

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИМЕНИ АКАДЕМИКА С.П. КОРОЛЕВА»

Лабораторная работа №6
Многопоточное вычисление интегралов функций

Выполнила:

Иванова Елизавета Вадимовна,
студентка группы 6301-030301D

Самара 2025

Задание 1. Численное интегрирование методом трапеций

В классе Functions реализован статический метод:

```
public static double integral(Function function, double leftX, double rightX, double dx)
```

- Метод разбивает отрезок $[a, b]$ на подынтервалы длиной dx (последний — остаточный).
- На каждом подынтервале вычисляется площадь трапеции.
- Добавлена проверка: если a или b выходят за границы области определения функции — выбрасывается исключение.
- Для функции $f(x)=e^x$ на $[0, 1]$ установлено: при $dx=10^{-8}$ достигается точность до 7 знака после запятой.

Задание 2. Последовательная реализация (nonThread)

- Создан метод nonThread(), формирующий 100 заданий:
 - функция: логарифм с основанием $\in [1, 10]$, $\in [1, 10]$,
 - $x_{\text{left}} \in [0, 100]$, $x_{\text{right}} \in [100, 200]$, $dx \in [0, 1]$.
- Все задания обрабатываются последовательно, без потоков.
- Вывод: 100 пар строк Source ... / Result

Задание 3. Простая многопоточная реализация (simpleThreads)

- Реализованы классы:
 - SimpleGenerator (генератор),
 - SimpleIntegrator (вычислитель).
- Оба потока синхронизируются на объекте Task через:
 - synchronized (task),
 - wait() / notify().
- Проблема: при первом запуске наблюдались:
 - NullPointerException (интегратор читал null),
 - "перемешивание" параметров (чтение половины данных от одного задания, половины — от другого).
- Решение:

Генератор ждёт, пока `function == null`, затем заполняет все поля.

Интегратор ждёт, пока `function != null`, читает всё, затем сбрасывает `function = null` и уведомляет.
- Результат: 100 корректных пар, без повторов и ошибок.

Задание 4. Реализация с собственным семафором (complicatedThreads)

- Создан класс SimpleSemaphore — бинарный семафор на основе wait()/notify().
- Изначально использовался один семафор, но это привело к гонке: оба потока конкурировали за захват, и данные перезаписывались.
- Решение: применена классическая схема Producer–Consumer с двумя семафорами:
 - empty — сигнал, что Task пуст и можно писать,
 - full — сигнал, что Task заполнен и можно читать.
- В main() семафор full изначально занят, чтобы интегратор ждал первую запись.

- Результат:
Строго поочерёдный вывод Source → Result,
все задания уникальны,
нет потери данных,
программа устойчива к изменению приоритетов

```
Source 24.58385180462691 124.8660622365318 0.3846302049389453
Result 24.58385180462691 124.8660622365318 0.3846302049389453 244.1416280450877
Source 57.3678229218528 179.89544058966723 0.07487709488409877
Result 57.3678229218528 179.89544058966723 0.07487709488409877 488.04663288725754
Source 6.046862009405995 168.24807096640006 0.5517195705105368
Result 6.046862009405995 168.24807096640006 0.5517195705105368 887.5069419530238
Source 16.236368111954256 157.52730489698214 0.5983869048228391
Result 16.236368111954256 157.52730489698214 0.5983869048228391 730.9787523980095
Source 66.46573944634846 108.27930245053463 0.9299376845459825
Result 66.46573944634846 108.27930245053463 0.9299376845459825 81.27573139333943
Source 19.812724088796017 198.31867120410402 0.15896002893423788
Result 19.812724088796017 198.31867120410402 0.15896002893423788 611.9970096618849
Source 57.02364178975232 106.82746309673779 0.7210429006781184
Result 57.02364178975232 106.82746309673779 0.7210429006781184 138.39026508819111
```

```
Source 5.5557983054842275 164.32205940854976 0.008841074315053765
Result 5.5557983054842275 164.32205940854976 0.008841074315053765 330.5865724857086
Source 78.6296896320326 117.66420138403662 0.14098420694166758
Result 78.6296896320326 117.66420138403662 0.14098420694166758 467.37376254178275
Source 23.574143761987166 182.6693756936316 0.7594429413991625
Result 23.574143761987166 182.6693756936316 0.7594429413991625 376.14512242719616
Source 81.21141408639096 138.3534830591498 0.20600990238977834
Result 81.21141408639096 138.3534830591498 0.20600990238977834 169.64038825421602
Source 95.24147705235964 135.0140389712242 0.7333766076145025
Result 95.24147705235964 135.0140389712242 0.7333766076145025 216.9503626037768
Source 96.20022982876011 161.58216133078028 0.1231126207037061
Result 96.20022982876011 161.58216133078028 0.1231126207037061 139.16781568117094
Source 85.7389887131078 169.22007793488487 0.2466920962234913
Result 85.7389887131078 169.22007793488487 0.2466920962234913 225.62434524822035
Source 90.66191439038305 199.82442350186165 0.36598517446279966
Result 90.66191439038305 199.82442350186165 0.36598517446279966 359.5656340118356
```

```
Process finished with exit code 0
```

Выводы

- Метод трапедий успешно реализован и протестирован.
- Примитивы wait()/notify() позволяют организовать чередование потоков, но требуют строгого контроля состояния (function = null).
- Использование собственного семафора обеспечивает надёжную синхронизацию.
- Схема с двумя семафорами (empty/full) — наиболее надёжное решение для задачи "один производитель — один потребитель".
- Все режимы работы (nonThread, simpleThreads, complicatedThreads) протестированы и корректны.